

Entrega Integrada do Challenge Clyvo Vet

Cloud Computing & DevOps | Java Advanced | Banco de Dados Relacional

Folha de rosto

Equipe:	Challenge Wise Clyvo
Aluno(s):	Preencher nomes completos em ordem alfabetica
RM:	Preencher RM dos integrantes
Data:	19/05/2026
Projeto:	PortalWeb Clyvo Vet com Spring Boot, Docker Compose e Oracle

Resumo da complementacao

A entrega foi complementada com os requisitos do documento 2TDS Fevereiro - Challenge 2026 - 1o Semestre: containerizacao em nuvem, API Java/Spring Boot com recursos avancados, modelagem e scripts Oracle, evidencias e checklist por disciplina.

Indice

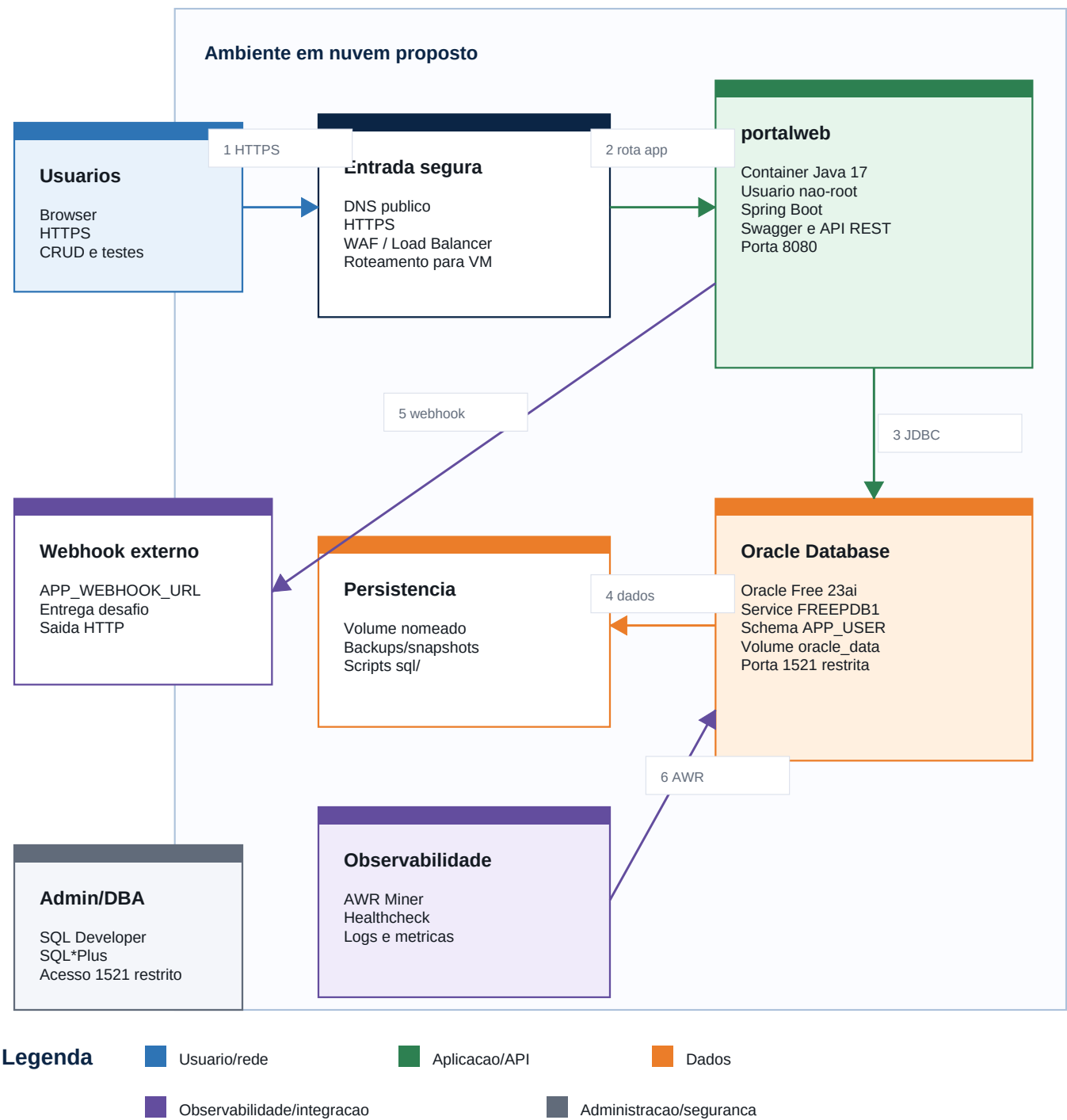
01	Folha de rosto, escopo e indice	1
02	Arquitetura macro em nuvem	2
03	DevOps Tools & Cloud Computing	3
04	Java Advanced	4
05	Banco de Dados Relacional	5
06	Diagramas de dominio e relacional	6
07	Capacidade Oracle pelo AWR Miner	7
08	Links, evidencias e proximos passos	8

Campos pendentes

Preencher GitHub, YouTube, nomes/RMs e anexar print real de remocao da VM quando a demonstracao em Azure for executada.

02. Arquitetura Macro em Nuvem

Fluxo de usuario, front-end, API, banco Oracle, VM/container, persistencia, observabilidade e administracao.



Equivalencia em nuvem: VM Linux Azure com Docker ou container gerenciado; Oracle em container/VM ou servico gerenciado, mantendo rede privada, volume persistente e acesso administrativo controlado.

03. DevOps Tools & Cloud Computing

Checklist baseado nos slides 22 a 26 do documento do Challenge.

Status	Requisito	Aplicacao no projeto
OK	CRUD GET/POST/PUT/DELETE	Endpoints /alunos e /api/v1 com tutores, pets e planos de cuidado.
OK	Persistencia Oracle containerizada	Servico oracle-db usa gvenz/oracle-free e volume nomeado oracle_data.
OK	Dockerfile e Compose	Dockerfile multi-stage Maven/Java 17 e docker-compose.yml com app + banco.
OK	Aplicacao em background	docker compose up -d executa portalweb e oracle-db em background.
OK	Usuario sem privilegio	Dockerfile cria usuario portal e executa USER portal no runtime.
OK	Volume nomeado do banco	oracle_data:/opt/oracle/oradata persiste dados Oracle.
OK	Azure CLI	Scripts cloud/azure-create-vm-docker.sh e azure-delete-resources.sh.
PENDENTE	GitHub publico	Preencher link real no PDF/README apos publicar repositorio.
PENDENTE	Video YouTube	Gravar demonstracao Azure CLI, Docker, CRUD e persistencia.
PENDENTE	Print remocao VM	Executar script de delete e anexar evidencia real de remocao.

Roteiro de demonstracao

Executar cloud/azure-create-vm-docker.sh, acessar <http://IP:8080>, testar Swagger/Postman, demonstrar dados persistindo no Oracle, executar cloud/azure-delete-resources.sh e registrar o print de remocao.

04. Java Advanced

Checklist baseado nos slides 27 a 33: Spring Boot, POO, JPA, REST, validacao, cache, DTOs, Swagger e testes.

Status	Requisito	Aplicacao no projeto
OK	Solucao no contexto Clyvo Vet	Modulo /api/v1 organiza tutores, pets e planos de cuidado para continuidade clinica.
OK	POO e entidades relacionadas	Tutor 1:N Pet e Pet 1:N PlanoCuidado mapeados com JPA.
OK	Coesao/desacoplamento	Controllers delegam para AlunoService e ClyvoVetService; DTOs isolam contrato HTTP.
OK	RESTful	Recursos, verbos HTTP, status 201/204 e rotas versionadas /api/v1.
OK	Bean Validation	Requests usam @NotBlank, @Email, @Past, @FutureOrPresent, @DecimalMin e @Size.
OK	Paginacao e ordenacao	Pageable e @PageableDefault em /alunos, /tutores, /pets e /planos-cuidado.
OK	Busca com parametros	Parametro busca e filtro status em endpoints de consulta.
OK	Cache	@EnableCaching, @Cacheable e @CacheEvict nos servicos.
OK	Tratamento de erros	GlobalExceptionHandler retorna ErrorResponse padronizado.
OK	DTOs	Requests/responses dedicados para Aluno, Tutor, Pet e PlanoCuidado.
OK	Swagger	springdoc-openapi em /swagger-ui/index.html.
OK	Postman/Insomnia	Colecao em documentos/PortalWeb-ClyvoVet.postman_collection.json.

05. Banco de Dados Relacional

Checklist baseado nos slides 34 a 39: modelagem, DDL, carga, logs, joins, LAG/LEAD e cursores.

Status	Requisito	Aplicacao no projeto
OK	Modelo descritivo	documentos/modelagem-relacional.md descreve estrutura e 3FN.
OK	DER/MER coerente com Java	T_TUTOR, T_PET e T_PLANO_CUIDADO coincidem com entidades JPA.
OK	DDL Oracle	sql/01_schema_oracle.sql cria tabelas, PKs, FKs, checks e indices.
OK	Carga por procedures	sql/02_procedures_seed.sql contem procedures parametrizadas por tabela.
OK	Tratamento de excecoes	Procedures tratam DUP_VAL_ON_INDEX, VALUE_ERROR e WHEN OTHERS.
OK	Tabela de logs	T_LOG_ERRO registra procedure, usuario, data, codigo e mensagem.
OK	Dados significativos	Seed inclui alunos, tutores, pets e cinco planos de cuidado.
OK	Joins/group/order	sql/03_reports_oracle.sql possui 3 consultas com joins, agrupamento e ordenacao.
OK	Linha anterior/proxima	Relatorio usa LAG/LEAD e mostra Vazio quando nao existir.
OK	Cursores explicitos	Quatro blocos anonimos com cursor explicito e tomada de decisao.

Observacao

Os scripts estao montados para Oracle e tambem sao montados no container em /container-entrypoint-initdb.d pelo docker-compose.yml.

06. Diagramas de Dominio e Relacional

Representacao textual dos relacionamentos para apoiar Java Advanced e Banco Relacional.

Diagrama de classes de entidade



DER / MER fisico Oracle

Tabela	Chave primaria	Relacionamentos/constraints
T_TUTOR	ID_TUTOR	UK DS_EMAIL; pai de T_PET
T_PET	ID_PET	FK ID_TUTOR -> T_TUTOR; peso > 0
T_PLANO_CUIDADO	ID_PLANO_CUIDADO	FK ID_PET -> T_PET; checks de categoria/status
T_LOG_ERRO	ID_LOG	Auditoria de erros das procedures
T_ALUNO	RM_ALUNO	CRUD legado do portal visual

Normalizacao

O modelo separa responsavel, pet e plano de cuidado em tabelas independentes. Atributos dependem da chave primaria da propria tabela e os relacionamentos sao controlados por chaves estrangeiras, atendendo ao minimo de 3FN solicitado.

07. Capacidade Oracle pelo AWR Miner

Resumo do arquivo AWR-Miner/awr-hist-1497016490-FREE-0-7.out usado para justificar dimensionamento inicial.

Metrica	Valor / interpretacao
Banco / DBID	FREE / 1497016490
Versao e arquitetura	Oracle Free 23.26.1.0.0, CDB com 1 PDB
Instancia e plataforma	1 instancia, Linux OS (AARCH64)
CPU e memoria	10 CPUs reportadas; cpu_count efetivo 2; memoria fisica 7,65 GB
Memoria Oracle	SGA ~1,5 GB, PGA ~0,3-0,4 GB, total ~1,8-1,9 GB
Disco Oracle	1,4 GB alocado; ~0,99 GB utilizado
Historico AWR	Snapshots 1 a 8, total de 8 snapshots, sem erro de coleta
Carga observada	CPU OS ~1,0% a 1,6%; AAS proximo de 0; workload baixo
SQL/aplicacao	Cliente JDBC Thin no schema APP_USER; acao SELECT identificada

Recomendacao de capacidade inicial

Como a carga medida e baixa, iniciar com 1-2 vCPU e 1-2 GB para o container da aplicacao. Para Oracle, usar baseline de 2 vCPU, 4-8 GB RAM e storage inicial de 20 GB com crescimento automatico, backups e snapshots. Escalar primeiro a aplicacao horizontalmente e revisar o banco com AWR apos testes de carga.

08. Links, Evidencias e Proximos Passos

Itens finais exigidos pela apresentacao para fechamento da entrega no portal FIAP.

Item	Valor / caminho
Repositorio GitHub	PREENCHER: link publico do repositorio
Video YouTube	PREENCHER: link do video com voz e qualidade 720p+
Print remocao VM/recursos	PENDENTE: anexar evidencia apos executar cloud/azure-delete-resources.sh
PDF final	arquitetura-cloud-portalweb-oracle.pdf
Colecao Postman	documentos/PortalWeb-ClyvoVet.postman_collection.json
Scripts Azure CLI	cloud/azure-create-vm-docker.sh e cloud/azure-delete-resources.sh
Scripts Oracle	sql/01_schema_oracle.sql, 02_procedures_seed.sql, 03_reports_oracle.sql
Swagger local	http://localhost:8080/swagger-ui/index.html

Como executar para validar

- Subir localmente: `docker compose up --build -d`
- Abrir a aplicacao: `http://localhost:8080` e Swagger: `/swagger-ui/index.html`
- Importar a colecao Postman e executar os endpoints de alunos, tutores, pets e planos.
- Na Azure, configurar `REPO_URL` e `ADMIN_IP_CIDR`, executar `cloud/azure-create-vm-docker.sh` e demonstrar a aplicacao pelo IP publico.
- Ao final, executar `cloud/azure-delete-resources.sh` e capturar print da remocao do Resource Group.

Nota final

Sem os links reais e o print de remocao, estes itens permanecem como pendencias operacionais, pois dependem de publicacao no GitHub/YouTube e execucao real na Azure.